

C++&Qt Day04 hw1 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.08

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 C++&Qt Day04 과제에서는 Qt의 QUdpSocket을 활용하여 두 컴퓨터가 UDP 방식으로 문자 메시지를 주고받을 수 있는 프로그램을 구현하였다.

프로그램의 UI 구성은 직접 진행하였으며, 프로그램을 구현하는 과정에서 C++ 및 Qt 코드 작성, 기존 코드와 UDP 기능의 결합, 빌드 오류와 통신 오류 해결 과정에서 AI의 도움을 받았다.

처음에는 새로운 채팅 프로그램을 UI부터 다시 구현하려 하였으나, 이전 Day03 과제에서 작성했던 천지인 키보드를 메시지 입력 기능으로 활용하기로 하였다. 천지인 키보드의 기존 한글, 영어, 숫자 및 기호 입력 기능은 그대로 유지하면서, 기존 Enter 버튼의 동작을 파일 저장 방식에서 UDP 메시지 전송 방식으로 변경하였다.

이 과정에서 기존 천지인 키보드 코드를 어떤 부분까지 유지해야 하는지, UDP 통신 기능을 어느 위치에 추가해야 하는지 판단하기 위해 AI에게 기존 mainwindow.h, mainwindow.cpp, main.cpp 코드를 제공하고 코드 구조에 대한 도움을 받았다.

2. AI 사용 부분

좋아. 그럼 지금부터는 기존 천지인 코드에 UDP를 붙이는 작업으로 간다.

현재 수정할 핵심은 src/mainwindow.cpp 야. 우선 mainwindow.cpp 전체 코드를 여기 붙여줘. 길어도 괜찮아.

내가 그 코드 기준으로:

1. 기존 천지인 입력/한글 조합 로직은 최대한 유지
2. 엔터 버튼 함수가 어디인지 찾기
3. 거기에 writeDatagram() 전송 추가
4. readyRead → udpRead() 수신 기능 추가
5. 필요한 mainwindow.h 변경
6. UI에 IP / Port / 채팅창만 추가

순서로 수정해줄게.

지금은 코드 먼저 보여줘. 기존 코드를 모르고 새로 짜면 전에 만든 천지인 동작을 괜히 망가뜨릴 수 있어.

📄 🗨️ ↶️ ↷️ ...

</> mainwindow.cpp
파일

</> main.cpp
파일

</> mainwindow.h
파일

좋아. 지금 코드 확인했어. 천지인 입력 로직은 그대로 두고, Enter 가 하던 "txt 저장"만 "UDP 전송"으로 바꾸는 방식이 가장 깔끔해.

현재 구성하는 부분은 on_enterButton_clicked() 에 UDP 송신을 붙이고, 새 udpRead() 함수로 수신하는 부분이야. 기존 천지인 문자 순환, 한글 조합, 한/영, 숫자-특수문자, Backspace 등은 건드리지 않을 거야. 현재 Enter 함수도 이미 입력 중인 문자를 확인한 뒤 committedText 를 검사하는 구조라 이 앞부분을 그대로 재사용할 수 있어. 📄 mainwindow

기존 Day03 천지인 키보드는 버튼을 이용하여 한글, 영어, 숫자 및 특수문자를 입력하고, Enter 버튼을 누르면 작성한 문장을 파일에 저장하는 방식으로 구현되어 있었다.

이번 과제에서는 기존 입력 기능을 그대로 유지하면서 Enter 버튼을 눌렀을 때, 작성된 문장을 상대방에게 UDP 로 보내도록 변경이 필요했다.

기존 코드의 구조가 길고 한글 조합과 관련된 함수가 여러개 존재하였기 때문에, 전체 코드를 AI 에게 제공하고 어떤 부분은 유지하고 어떤 부분만 수정해야하는지 도움을 받았다.

AI 의 도움을 통해 기존의

```
commitCurrentCharacter();  
comminkoreanSyllable();
```

등의 천지인 문자 확정 과정은 그대로 유지하고, 기존 파일 저장 부분을 UDP 전송 코드로 변경하였다.

이런 방식을 이용하여 이미 구현되어 있던 천지인 키드 기능을 다시 작성하지 않고 새로운 기능에 재사용할 수 있었다.

결과적으로 기존 코드에서 바뀐 건 딱 이것

천지인 입력 로직
↓
전혀 안 건드림
↓
committedText
↓
Enter
↓
기존: output.txt 저장
↓
변경: UDP 전송
↓
입력 초기화

main.cpp 는 수정할 필요 없어. 지금 QApplication → MainWindow 생성 → show() → exec() 로 정상적인 Qt 실행 구조야. [main](#)

UI 5개 추가하고 위 코드 수정한 다음 **일단 빌드해봐**. 오류가 뜨면 그 오류 화면 기준으로 잡으면 돼.

출처

bindButton → QPushButton이건 어따쓰든겨

bindButton 은 "수신 시작" 버튼이야.
즉 이 버튼을 누르면:
내 Port 번호 확인
→ udpSocket->bind(...)
→ 그 Port로 들어오는 UDP 메시지를 받을 준비
이 함수랑 연결돼:

```
C++  
void MainWindow::on_bindButton_clicked()  
{  
    ...  
    udpSocket->bind(  
        QHostAddress::AnyIPv4,  
        localPort  
    );  
}
```

예를 들어 내가
내 Port = 9999
로 입력하고 bindButton 을 누르면,
9999번 포트로 오는 UDP 데이터 수신 시작
이 되는 거야.
UI에서는 위쪽에 이렇게 하나 두면 돼:
내 Port [9999] [수신 시작]
여기서 [수신 시작] 버튼의 objectName이
bindButton
이야.
근데 원하면 이 버튼 자체 없애고 프로그램 실행하자마자 자동으로 9999번 포트를 bind하게 만들 수도 있어. 그게 UI는 더 단순해져.

뿐만 아니라 코드를 이해하며 궁금했던 점들을 AI 에게 질문하기도 하였다.

3. 깨달은 점

처음에는 QUdpSocket, bind(), writeDatagram() 등의 함수가 각각 무엇을 의미하는지 잘 알지 못했지만, 코드를 작성하면서 각 함수의 역할을 설명받고 실제 실행 결과와 연결하여 이해할 수 있었다.

특히 Qt의 Signal / Slot에 대해서 이해가 더 깊어졌다. 이전 과제에서는 버튼 클릭이나 QTimer를 이용할 때 Signal / Slot을 사용하였지만, 이번에는 UDP 데이터가 도착했을 때 발생하는 readyRead Signal을 이용하였다. 이를 통해 Signal / Slot이 UI 입력뿐만 아니라 네트워크 이벤트를 처리하는 데에도 사용된다는 것을 알게 되었다.

AI를 사용하면서 가장 중요하다고 느낀 점은 코드를 단순히 복사하여 사용하는 것보다 오류가 발생했을 때 해당 코드가 어떤 역할을 하는지 이해하면서 수정하는 것이 중요하다는 점이다. 이번 과제에서는 AI가 제시한 코드를 바로 사용했을 때도 여러 빌드 오류나 네트워크 설정 문제가 발생하였다. 이러한 문제를 하나씩 확인하면서 코드를 함께 이해하며 작성해나아가야 프로그램을 완성할 수 있다는 것을 알게 되었던 것 같다.